Learning objectives

- Reason about mutual exclusion, progress, and bounded waiting
- Use semaphores for ordering and backpressure
- Identify deadlock, starvation, priority inversion, and lost wake-ups

Central question. How can arbitrary interleavings preserve shared state while allowing progress and bounded waiting?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Race invariants and the complete counter trace

Producer executes R1=counter; R1=R1+1; counter=R1. Consumer executes R2=counter; R2=R2−1; counter=R2. The net serial effect is zero, but interleavings can leave 4,5,6 from initial 5. Treating counter++ as one step hides the conflict. This is an abstract interleaving model; unsynchronized non-atomic conflicting C/C++ accesses can be a data race with undefined behavior, not a guarantee of only these results.

Event	Producer	Consumer	Counter	Observation
T0	R1=5		5	Old value loaded
T1	R1=6		5	Only register changed
T2		R2=5	5	Same old value
T3		R2=4	5	Only register changed
T4	counter=6		6	Producer stores
T5		counter=4	4	Consumer overwrites

Swapping T4/T5 leaves 6; the intended result is 5. Mutual exclusion covers only agreed regions. Visibility, reordering, non-atomic widths, and failure retries need the same synchronization contract.

Race invariants and the complete counter trace (continued)

Identify all shared mutable locations, state cross-field invariants such as $0 \le size \le capacity$ or readers=0 or writer=0, identify operations temporarily breaking them, and define which observers must be excluded plus acquire/release visibility. Atomic stores to a list's next and size do not make their combined update indivisible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safety, progress, and bounded waiting

Each of n processes P0…P(n−1) has entry, critical, exit, and remainder sections.

Requirement	Definition	Failure
Mutual exclusion	While Pi is in its critical section, no other participant is in its critical section	Concurrent writers break one invariant
Progress	If the section is empty and contenders exist, only non-remainder contenders participate in selection, and selection cannot be postponed indefinitely	No contender is selected
Bounded waiting	Every requester has a finite bound on other entries allowed before its admission	Global progress while one requester starves

An unfair exclusive lock can repeatedly admit high-priority arrivals without bounding a lower-priority wait. Progress with exclusion still permits indefinite overtaking. Conversely, admitting everyone can be fast yet violate exclusion. Safety must cover every reachable interleaving; progress and fairness need separate arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safety, progress, and bounded waiting (continued)

For enter_i/leave_i, prove exclusion at arbitrary relative speeds; finite-step admission progress when free with contenders; a finite overtaking bound per requester; and the atomicity, ordering, and failure recovery of entry/exit themselves. An overtaking bound counts other admissions and is not a wall-clock bound. Eventual entry also needs scheduling and termination assumptions. A finite sample of schedules can find counterexamples, not prove arbitrary n and speeds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Peterson's complete protocol

There are exactly two processes, j=1−i, atomic flags initially false, and shared turn.

```
P_i:
  flag[i] = true
  turn = j
  while (flag[j] && turn == j) { wait }
  critical_section_i
  flag[i] = false
  remainder_i
```

Flag means competing. Turn resolves simultaneous contention, not permanent lock ownership. The slogan that the last turn writer always loses omits required contention premises.

Assume atomic sequentially consistent reads/writes. Suppose the two current critical-section attempts overlap. Let W0 be P0's turn=1 and W1 P1's turn=0. If W0<W1, P0's flag0=true preceded W0 and W1. Before P0 leaves, flag0 remains true; neither participant writes turn again inside this attempt. P1 therefore faces flag0=true and turn=0 when it would need to pass, making its while condition true, a contradiction. If W1<W0, symmetry blocks P0. Thus overlapping critical sections are impossible. This compares the current overlapping attempts' final turn writes, not arbitrary past iterations.

Peterson's complete protocol (continued)

For progress, one contender sees the other's false flag and enters. With both competing, turn selects one side whose while is not blocked by turn. A participant already inside eventually clears its flag on exit. For bounded waiting, blocked Pi has flag[j]=true and turn=j. Pj does not change turn in its critical section. After leaving it clears flag; any new attempt first writes turn=i, giving Pi priority. From the point Pi has completed flag[i]=true and turn=j, Pj can enter at most once before Pi, assuming relevant threads continue taking steps and critical sections terminate. Counting from the earlier flag write alone does not give this one-entry bound.

The proof needs shared order/visibility. Use corresponding atomics with memory_order_seq_cst, or a proved equivalent fencing protocol. Renaming ordinary variables flag/turn supplies no such proof.

Bakery algorithm: equal numbers still have an order

There are n fixed participants, each with at most one outstanding request. `choosing[i]` starts false and `number[i]` starts 0, meaning no request. For this teaching proof, all shared accesses are sequentially consistent atomic operations and ticket numbers do not overflow. Lamport's original paper considers a weaker read/write model.

```
choosing[i] = true
number[i] = 1 + max(number[0..n-1])
choosing[i] = false
for each j != i:
  while choosing[j]: wait
  while number[j] != 0 and (number[j],j) < (number[i],i): wait
critical section
number[i] = 0
```

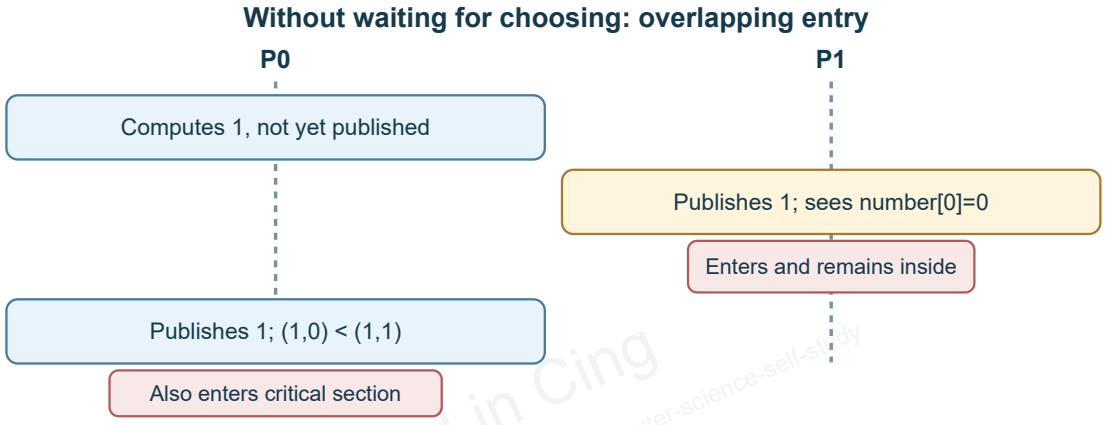
Pairs use lexicographic order: compare the number first, then the participant ID. The max scan is not an indivisible snapshot: two participants can both observe maximum 0 and choose 1. If P0 and P1 finish choosing with pairs (1,0) and (1,1), P0 precedes P1; P1 waits until P0 clears its number.

Bakery: why the choosing flag matters

`choosing[j]` means Pj is selecting a number. An observer waits for that selection to finish before comparing the published number. Without that step, initially all zero:

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bakery: why the choosing flag matters (continued)



Fix: P1 waits for choosing[0] to finish before comparing

The correct protocol holds P1 at `choosing[0]=true` until P0 publishes its choice, after which (1,0) has priority. The proof uses completed selection and pair order; it cannot assume tickets are always different.

After Pi completes choosing, at most n−1 other requests are already present or choosing. A participant that requests again while Pi's number remains published must choose after it. To conclude eventual entry, also assume selection and critical sections finish and relevant threads continue to be scheduled; this is not a wall-clock bound.

Bakery: mutual exclusion for general n

Suppose the current requests of Pi and Pj overlap in their critical sections. Pi has already published its number before checking Pj and keeps it unchanged until Pi exits.

If Pj had published its current nonzero number when Pi checked, Pi waits for `choosing[j]=false` and can pass only if `(number[i],i) < (number[j],j)`. If Pj has not yet checked Pi, the strict total order forces Pj to wait. If Pj checked and passed before Pi began its current request, Pj's number was already published and remains so until exit; Pi's later max scan sees it and selects a larger number, so Pi cannot pass. If selections overlap, `choosing` makes the check wait until the other selection is published, after which the pair order applies.

Another interleaving is that Pi reads `choosing[j]=false`, Pj then starts choosing, and Pi subsequently reads `number[j]=0`. These two reads are not an atomic snapshot. But Pj started after Pi had published its number; if the critical sections overlap, Pi's nonzero number remains valid while Pj scans, so Pj chooses a strictly larger number and waits for Pi to clear it. If Pj was already choosing when Pi read `choosing[j]`, Pi waits for that selection to complete. The flag does not stop a new selection from starting after Pi's check.

Both cases contradict overlap. Under the stated SC, nonoverflowing-number, and one-outstanding-request assumptions, mutual exclusion holds for arbitrary n.

Atomic hardware and spinning

Test-and-set atomically returns old L and sets true. CAS(&L,0,1) sets 1 only when L was 0 and returns the old value whether it succeeds or not.

```
acquire(L):
  repeat old = test_and_set(L)
  until old == false
release(L):
  L = false
```

One acquire sees false and owns the lock; others see true and retry. Owner-only release with proper publication preserves exclusion. Basic test-and-set does not bound waiting. Queueing, such as a circular waiting array with explicit successor handoff, adds fairness.

Spinning avoids sleep/wakeup and context costs for short sections. Long sections, one core, or a spinner competing with its owner can waste CPU. An interrupt handler spinning on a lock held by the interrupted task can prevent that owner resuming, producing livelock/deadlock. Interrupt contexts need controlled non-sleeping primitives and suitable interrupt/spin rules.

Atomic hardware and spinning (continued)

Instruction indivisibility does not alone provide language-level acquire/release. Later owners must see prior protected writes; writes before release must remain before publication; compilers must not move critical accesses outside the lock. Use atomic acquire/release semantics or corresponding fences. CAS alone does not solve ABA, reclamation, or lock-free progress.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mutex ownership and ordering proof

Every participating critical section uses acquire(m), protected work, release(m). Acquisition/release transitions are atomic. A mutex usually requires its owner to release it; a semaphore need not enforce ownership. Error, cancellation, and exception paths depend on that distinction. Blocking mutex contenders wait; spinners continue consuming CPU and require short sections with a runnable owner.

Assign a strict global lock order, ord(A)=1, ord(B)=2. Acquire a new lock only above every held lock. Any resource-wait cycle would require ord(A)<…<ord(A), a contradiction. In particular holding B while acquiring A violates ord(B)<ord(A). Callbacks, condition waits, upgrades, exceptions, and cancellation must obey the same rule. Sort dynamic locks by a stable address or unique ID, not transaction argument order such as from/to accounts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permit abstraction and signed implementation

A resource semaphore counts permits; a binary abstraction has 0 or 1 free permits. A signed blocking implementation also encodes waiters. Counter and queue actions must be atomic with blocking/wakeup.

```
busy_wait(S):
  loop: old = atomic_load(S)
        if old > 0 and CAS(S, old, old-1) succeeds: return
busy_signal(S): atomic_fetch_add(S,1)

wait(S):
  S--
  if S < 0:
    enqueue current on S.queue
    block current
signal(S):
  S++
  if S <= 0:
    dequeue one waiter
    wake it
```


Permit abstraction and signed implementation (continued)

The busy-wait notation requires indivisible successful test/decrement, not two racing ordinary operations. Negative signed state represents waiters, not negative resources. At quiescent boundaries, free=max(S,0), queued=max(−S,0). A handed-off permit is allocated even if its recipient has not received CPU; newcomers cannot take it. Resource semaphores preserve free+allocated=capacity. Event semaphores need not have fixed capacity.

For capacity 1: T0 wait 1→0 obtains the permit; T1 wait 0→−1 queues/sleeps. T0 signal−1→0 wakes T1 because the incremented value≤0 and hands it the permit. T1 resumes after its original wait without decrementing again. T1 signal0→1 leaves a free permit because nobody waits. Waking only on0→1 strands T1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permit abstraction and signed implementation (continued)

For capacity 3, empty=3, full=0, mutex=1:

Completed operation	Empty	Full	Buffer
Initial	3	0	[]
Put A	2	1	[A]
Put B	1	2	[A, B]
Put C	0	3	[A, B, C]
Get A	1	2	[B, C]
Put D	0	3	[B, C, D]

Producer uses wait(empty), wait(mutex), update, signal(mutex), signal(full). Consumer first waits full. Holding mutex while awaiting capacity can prevent the consumer releasing capacity.

If P0 takes S then Q and P1 Q then S, each initially1, P0 can hold S and P1 Q before waiting forever for the other. Starvation instead leaves some work progressing while a waiter is indefinitely skipped; LIFO arrivals can do that. FIFO removes one starvation pattern but not incorrect pairing.

Permit abstraction and signed implementation (continued)

For priority inversion, low-priority L owns H's needed resource while medium M preempts L. In the single-lock example, inheritance temporarily raises L; after release its effective priority is recomputed. With multiple locks or chained donations it need not revert to the base priority. The historical Mars Pathfinder VxWorks incident illustrates this mechanism.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Classical coordination protocols

Empty prevents producing into a full buffer, full prevents consuming an empty one, and mutex protects compound slot/head/tail/size updates. Capacity waits occur outside mutex. Otherwise producer waits for consumer capacity while consumer waits for producer's mutex. The occupancy safety condition is $0 \le \text{items} \le n$. General capacity accounting is $E+F+P+C=n$. E and F are the nonnegative, unreserved free permits of empty and full. P counts producers already granted an empty permit that have not yet signaled full; C counts consumers already granted a full permit that have not yet signaled empty. A permit handed directly to an awakened waiter counts as its P or C reservation even before it runs.

Every atomic protocol transition preserves the sum. Successful producer wait moves E to P; producer signal(full) moves P to F, or directly to a consumer's C on handoff. Consumer wait moves F to C; signal(empty) moves C to E or directly to another producer's P. Writing or removing the item does not change this capacity classification. With capacity 3 and an empty buffer, $(E,F,P,C)=(3,0,0,0)$. After only one successful wait(empty), it is $(2,0,1,0)$, so $E+F=2$, even before insertion. $E+F=n$ holds at quiescent boundaries with no reservations, $P=C=0$.

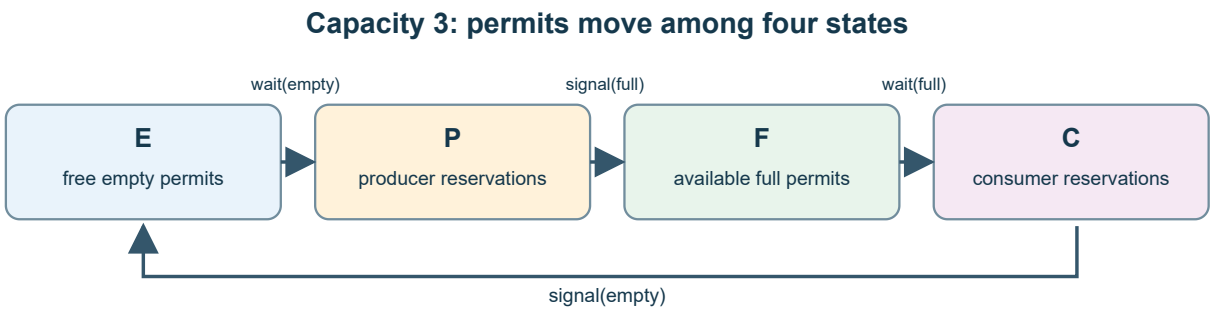
Classical coordination protocols (continued)

E/F are not signed semaphore raw counters. A negative raw value also encodes queued waiters and cannot be substituted into permit conservation. Raw zero can coexist with an outstanding handoff grant. Modify the structure under mutex, publish full only after insertion, and publish empty only after removal. These rules preserve occupancy safety; F need not equal the number of stored items during an operation.

Reader-preference allows new readers while writers wait, but not while a writer owns access; writers may starve. Writer-preference stops new readers after a writer starts waiting; readers may starve. Both allow concurrent readers when permitted. In the first protocol, read_count is protected by an owner-bound mutex; the first reader waits on rw_sem and the last reader signals it. Since they can be different threads, rw_sem is an ownerless binary semaphore, not a POSIX owner-bound mutex. A lock protects its actual state, not everything its name suggests.

Five philosophers each take chopstick[i], then chopstick[(i+1)%5], eat, release right then left. All can hold left and wait right, producing a five-edge cycle. Alternatives limit simultaneous contenders to four, acquire both atomically when available, or use opposite acquisition orders for odd/even philosophers. They exclude this deadlock pattern without automatically proving starvation freedom; waiter selection still matters.

Capacity conservation with in-flight reservations



Direct handoff: P can transfer to C; C can transfer to P

Event	E	F	P	C	Meaning
Empty buffer	3	0	0	0	Three unreserved empty slots
Producer succeeds in wait(empty)	2	0	1	0	Slot reserved, item not yet published
Producer writes and signals full	2	1	0	0	One item available
Consumer succeeds in wait(full)	2	0	0	1	Item reserved; slot not yet released
Consumer removes and signals empty	3	0	0	0	Buffer empty again

Every row has E+F+P+C=3. Cancellation must restore or complete any outstanding reservation.

Wait for a predicate, not a notification ticket

Use Mesa semantics, one mutex `m`, condition variable `not_empty`, and an initially empty shared queue `q`. P inserts A once. C1 and C2 each call `get()` once. Every queue mutation and `q.empty()` check holds the same mutex.

```
get():                          put_A():
  lock(m)                         lock(m)
  while q.empty():                q.push_back(A)
    wait(not_empty, m)            signal(not_empty)
  x = q.pop_front()               unlock(m)
  unlock(m)
  return x
```

Wait combines lock release and wait registration atomically relative to another thread acquiring m and notifying; successful return reacquires m. Signal does not transfer the mutex or reserve A for the waiter. With no waiter, no permit is accumulated.

Notification does not determine the next lock owner

Time flows down; P inserts one A; no spurious wakeups in this trace.

	Consumer C1	Producer P	Consumer C2	mutex / queue
T0	wait: unlock + enqueue			m=—, q=[]
T1		lock; enqueue A		m=P, q=[A]
T2	notified; no lock yet	signal C1; keep m		m=P, q=[A]
T3		unlock →	lock; take A; unlock	m=—, q=[]
T4	relock; q is empty			m=C1, q=[]
T5	while → wait again			m=—, q=[]

T3 orders P's unlock, C2's lock and removal of A, then C2's unlock. The mutex is not assumed FIFO. C1's T4 wait return owns m, but the predicate is false again. No spurious wakeup occurs in this counterexample: `if` would allow an empty pop; `while` sends C1 back to waiting at T5.

Track three distinct forms of state

CV holds condition waiters. E contains notified waiters awaiting m, **not a kernel ready queue**. Notification changes eligibility, not ownership or predicate truth. States follow each event.

Event	m owner	q	CV	E
T0 C1 waits	None	`[]`	`{C1}`	`{}`
T1 P inserts A	P	`[A]`	`{C1}`	`{}`
T2 P signals C1	P	`[A]`	`{}`	`{C1}`
T3 C2 removes A and unlocks	None	`[]`	`{}`	`{C1}`
T4 C1 reacquires	C1	`[]`	`{}`	`{}`
T5 C1 waits again	None	`[]`	`{C1}`	`{}`

Lost wakeup comes from a gap before waiting

Broken: separate unlock and wait

1. C1: sees empty, unlocks; not yet queued
2. P: inserts A, signals; nobody waiting
3. C1: queues and sleeps too late; q=[A]

Data exists, but C1 sleeps; signals do not accumulate.

Correct: atomic wait contract

1. C1: sees empty; wait unlocks + registers
2. P: locks, inserts A, signals, unlocks
3. C1: relocks, rechecks, then takes A

If P arrives first, C1 sees A and skips wait.

The left side is a broken abstract wait implementation: unlock first, register and block later. It is **not legal `pthread_cond_wait` usage**. P inserts and signals in the gap while CV is empty; C1 then sleeps despite queued data. Atomic wait excludes that gap. If P completes first and no other consumer removes A, C1 sees a nonempty queue under m and skips waiting. If A has already been consumed, C1 correctly waits again.

Prove safety and liveness separately

Safety. Every pop checks `!q.empty()` while holding m. No other thread can acquire m and change q between this check and removal. Wait return guarantees reacquisition, not predicate truth.

Liveness. The while loop prevents an invalid removal, but does not promise data for every consumer. There is only one A; once C2 consumes it, C1 may legitimately continue waiting. Scheduler fairness, mutex fairness, and future arrivals require separate assumptions.

If broadcast wakes two consumers for one A, the first owner takes A; the other returns to the while loop. Waking more threads creates neither data nor a bounded-waiting guarantee.

Handoff contracts: Hoare, Mesa, and signal-and-exit

The trace uses Mesa semantics: the signaler continues, and the waiter reacquires later. Hoare's handoff-time condition guarantee must not be imported into that model.

Handoff	Lock/execution after signal	Waiter reasoning
Hoare signal-and-wait	Immediate handoff; signaler waits	May rely on a predicate established before signal only under the immediate-handoff model without spurious wakeups or timeouts
Mesa signal-and-continue	Signaler retains the lock; waiter reacquires later	Recheck the predicate
Concurrent Pascal signal-and-exit	Signaler leaves, then hands off	Language-defined

Historical synchronization environments

Historical case	Mechanism	Boundary
Windows 7 kernel	SMP spinlocks for short global-resource sections	At the applicable IRQL ordinary thread scheduling does not preempt the holder; higher-IRQL interrupts may still occur
Windows 7 user services	Dispatcher mutex, semaphore, event, timer	Signaled/nonsignaled states; mutex grants one owner
Linux 2.6 onward textbook snapshot	Preemptible kernel, atomic_t, mutex, spinlock, semaphore	Short SMP spin regions; preempt_count>0 prevents safe preemption
Solaris textbook snapshot	Adaptive mutex, RW lock, turnstile	Spin while holder runs, otherwise sleep; turnstile supports inheritance
Pthreads/POSIX	Mutex, CV, RW lock; POSIX SEM semaphores	API determines waits, wakeups, errors

The textbook Windows critical-section object first spins in user mode and uses kernel assistance under prolonged contention. The cited uniprocessor Linux case replaces ordinary SMP-style spinning with preemption control. Solaris selects spin/sleep according to holder execution. These are versioned cases, not all-current-system claims.

Historical synchronization environments (continued)

Before substitution, identify user/kernel/interrupt context, scheduling entity, ownership and process-sharing support, fairness/inheritance/cancellation/wakeup order, and versioned memory ordering. Similar names do not establish equivalence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Alternatives and their remaining boundaries

A memory transaction makes a sequence of shared accesses all-or-nothing. Commit publishes it; abort rolls it back, excluding partial observation. For an example with (x,y)= (100,40), a successful atomic transaction x=x−25; y=y+25 commits (75,65), while an aborted attempt publishes neither change. Conflicting read/write sets can force a retry; isolation applies to observers following the same transaction contract. Aborting can avoid lock-order deadlock but still permits starvation and must handle I/O, irreversible effects, and retries. STM instruments accesses through compiler/runtime; HTM uses coherent-cache mechanisms. Capacity and failure paths differ.

Within the same contention group, OpenMP critical regions of the same name exclude one another; unnamed critical regions share one unspecified name, while different names do not exclude one another. The programmer still identifies shared conflicts. Immutable functional values avoid in-place writes and permit sharing old values, but files, networks, mutable databases, and mailboxes remain shared resources. Message copying/ownership transfer reduces shared mutation without eliminating capacity, ordering, backpressure, or retry coordination.

Choosing a synchronization primitive

Need	Required model	Check
One critical section	Length, sleepability, ownership	Paired acquire/release
Fixed resources	Capacity, initial permits, authorized signalers	No excess signal
Predicate wait	Variables, notification, recheck	One protecting mutex
Multiple locks	Global order, dynamic key, cancel paths	Every acquisition follows order
Readers/writers	Preference/fairness rules	Reader/writer starvation

Trace modes, owned resources, waiters, and predicates. A successful finite test can reveal a bug but does not prove correctness for all schedules.

Chapter conclusions

- Critical-section requirements separate safety, progress, and fairness
- Semaphore and monitor errors often arise from predicates and wait order
- Classical proofs require their stated memory-model assumptions

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra's Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study